



موّردي MY SUPPLIER
Build for Less

Developer Handover — Technical Guide

My Supplier (موّردي) · Building-materials price comparison, RFQ & supplier bidding platform

Web + dashboards + iOS/Android app

Repository: <https://github.com/invetechs1/My-supplier-> (branch `main` , commit `7076156`)

Version: 1.2 · Date: 17 September 2026 · Prepared for: the development team taking the platform to production

Owner: Bassir Technology (invetechs1)

1. ملخص تنفيذي (Arabic summary)

ما هو النظام: منصة "موّردي — Build for Less" تتيح لأي شخص في قطاع الإنشاءات معرفة سعر أي مادة بناء من كل الموردين في مكان واحد، وإرسال طلبات تسعير تصل فوراً للموردين المسجلين ليقدّموا عروضهم، ثم الترسية والشراء والدفع الآمن (حجز المبلغ لدى المنصة وتحويله للمورّد بعد التسليم). النظام يشمل: واجهة برمجة خلفية (API)، موقعاً إلكترونياً مع لوحات للمشترى والمورّد والإدارة، وتطبيق جوال iOS وAndroid.

ما تم إنجازه: النظام كامل الوظائف ومختبر (17 سيناريو اختبار شامل ناجحة، الموقع مُجرّب في متصفح حقيقي، صورة Docker جاهزة للنشر تُبنى تلقائياً على GitHub). كل التكاملات الخارجية (بوابة الدفع Moyasar، رسائل Unifonic/Twilio، البريد SMTP، الإشعارات Expo، التخزين S3) مبنية وتعمل في وضع التطوير بمزوّدات بديلة، وتُفعّل في الإنتاج بمتغيرات بيئة فقط.

ما المتبقي على المطوّر: (1) النشر على خادم بنطاق واسم و HTTPS وربط الحسابات الحقيقية (Moyasar، Unifonic، SMTP، متاجر التطبيقات) والتحقق منها بعملية حقيقية واحدة على الأقل، (2) تشغيل تطبيق الجوال على أجهزة حقيقية وبناءه ورفعها للمتاجر، (3) بنود تحسين مرتّبة حسب الأولوية في القسم 7 (مثل التخزين الخاص للمستندات، البحث النصي، فوترة زاتكا المرحلة الثانية، الدفع الآجل، فرق العمل للمشترين). التفاصيل الفنية لكل بند مع الملفات المعنية ومعايير القبول المذكورة بالإنجليزية أدناه.

2. What the system does

Buyers (contractors, project owners): search any material → see every supplier's price side by side (registered suppliers + external reference sources) with best/average/median, 90-day history and 30-day change; price alerts; buy directly or build a quote list → RFQ (also by importing a bill of quantities from Excel); receive ranked bids with market reference; award in one click → order → escrow payment (mada, cards, Apple Pay, STC Pay, bank transfer) → delivery tracking → tax invoice → rating; disputes.

Suppliers: register, upload compliance documents and get verified, manage a live price list per city (or bulk CSV import), receive matching RFQs, bid, confirm and deliver orders, receive payouts net of the platform fee, see win rate and revenue.

Platform admin: KPIs and GMV, supplier verification and plans, users, categories, external price feeds, data quality, finance (payments, bank-transfer confirmation, refunds, payouts), notification delivery log, scheduled jobs, document review, disputes.

8.9k

lines of app code

17

end-to-end tests
(green)

~95

API endpoints

3

clients: web,
dashboards, mobile

3. Architecture & repository layout

Layer	Stack	Folder
API + database	Python 3.12, FastAPI, SQLAlchemy 2, Pydantic 2, JWT (HS256), Alembic; SQLite in dev, PostgreSQL in prod	backend/

Website + buyer/supplier/admin dashboards	React 18, TypeScript, Vite, react-router; Arabic RTL + English; no UI framework (plain CSS with brand tokens)	web/
Mobile (iOS + Android)	Expo SDK 53, React Native 0.79, React Navigation, expo-notifications	mobile/
Deployment	Multi-stage Dockerfile (builds web, serves it with the API), docker-compose.prod.yml (Caddy HTTPS + API + PostgreSQL), GitHub Actions (tests, web build, image → GHCR)	backend/Dockerfile, docker-compose*.yml, deploy/, .github/
Docs & brand	README, DEPLOYMENT, ARCHITECTURE, API, INTEGRATIONS, BUSINESS_MODEL, ROADMAP, brand guide + assets	docs/, brand/

backend/app/	
main.py	FastAPI app, CORS, rate-limit middleware, Sentry, jobs thread, /uploads + SPA hosting
config.py	every setting, all overridable by environment variables
models.py	users, suppliers, categories, products, offers, price history, price sources, RFQs, bids,
	orders, reviews, notifications, price alerts, audit, payments, payouts, invoices, OTP codes,
	device tokens, notification deliveries, job runs, supplier documents, disputes
routers/	auth · catalog · market · suppliers · rfq · orders · payments · notifications · admin
services/	pricing (aggregation, index, trends) · ingestion (CSV/JSON feeds) · matching · notify
	channels (email/SMS/WhatsApp/push outbox) · otp · payments (mock/Moyasar, escrow, invoices)
	jobs (scheduler) · storage (local/S3) · boq (BOQ import, Excel export)
seed.py	Saudi materials catalog (58 products, 15 categories) + demo
suppliers/prices/history	
backend/tests/	pytest end-to-end suites (test_platform, test_payments_otp_notify, test_v12_uploads_boq_disputes)
backend/migrations/	Alembic (2 revisions)
web/src/	api.ts (typed client) · i18n.tsx · auth.tsx · components/ · pages/{public, buyer/, supplier/, admin/}
mobile/src/	api.ts · store.tsx · i18n.ts · theme.ts · push.ts · screens/

4. Feature inventory (what is built and verified)

Area	Included	Status
Auth & accounts	Email/password, JWT, roles buyer/supplier/admin, profile, password change, OTP (SMS/WhatsApp/email) for verified registration, passwordless phone login, password reset, phone verification, notification preferences, rate limiting	Done, tested

Catalog & pricing	Category tree, products (SKU, brand, unit, spec), offers per supplier/city with VAT normalisation, best/avg/median, supplier counts, 90-day history, 30-day change, market index, trending, compare, brands/cities, price alerts, product images	Done, tested
External price sources	CSV/JSON feeds by URL or upload, external "reference" suppliers, scheduled fetching, admin management	Done, tested
Suppliers	Profile, verification, plans, price list CRUD + CSV import, dashboard KPIs, logo, compliance documents with admin review, IBAN	Done, tested
RFQ & bidding	Multi-item RFQs (products or free text), public/invited visibility, category+city matching, supplier notifications, bids with items/delivery/terms (resubmit, withdraw), ranked comparison with market reference, award → order, close/expire, BOQ Excel/CSV import with product matching, Excel export of comparison	Done, tested
Orders	Direct orders from offers, awarded orders, state machine (pending → confirmed → in_delivery → delivered, cancel), reviews → supplier rating, disputes with admin resolution/refund	Done, tested
Payments	Escrow: checkout (mock gateway in dev; Moyasar hosted invoice in prod with webhook + API re-verification), bank transfer with admin confirmation, platform fee, release on delivery → payout, refunds, ZATCA phase-1 tax + fee invoices with QR (JSON + printable HTML), buyer/supplier/admin finance views	Done, tested (mock); Moyasar live call not exercised
Notifications	In-app + outbox to email (SMTP), SMS (Unifonic/Twilio), WhatsApp (Twilio/Meta), push (Expo); retries, delivery log, device registration	Done, tested (console providers); live providers not exercised
Jobs	Outbox sender, RFQ expiry, price alerts, feed fetching, payment expiry; in-process worker + cron endpoint; admin run/monitor	Done, tested
Admin	Dashboard KPIs/GMV/series, users, suppliers, categories, sources, data quality (stale/outliers), finance, deliveries, jobs, documents, disputes, audit log	Done, tested
Web app	All of the above with Arabic RTL/English, brand "Build for Less", responsive, rendered in Chromium end-to-end (login, pay, OTP, BOQ import, admin)	Done, browser-verified
Mobile app	Search/compare, product detail + direct order, quote list → RFQ, RFQ list/detail with bidding (supplier) and awarding (buyer), orders + pay (hosted checkout / bank transfer), OTP registration and phone login, notifications, push registration, supplier price list and payouts, brand icon/splash	Typechecked only — not run on a device
Deployment	Docker image (built by CI, published to GHCR), prod compose with HTTPS, migrations on start, deploy/backup scripts, systemd alternative, go-live guide	Done; not yet deployed to a real server

5. Running, testing, demo accounts

```
# API
cd backend && python3 -m venv .venv && source .venv/bin/activate && pip install -r
requirements.txt
uvicorn app.main:app --reload --port 8000          # http://localhost:8000 · docs at /docs
python -m pytest -q                                # 17 tests, ~6 s

# Web
cd web && npm install && npm run dev                # http://localhost:5173 (proxies /api →
:8000)
npm run build                                       # → web/dist, served by the API at /

# Mobile
cd mobile && npm install --legacy-peer-deps && npx expo start  # Expo Go on a phone; dev
API = http://<your-ip>:8000
```

Role	Email	Password
Admin	admin@mysupplier.sa	Admin@2026
Buyer	buyer@demo.sa	Demo@2026
Suppliers	supplier1@demo.sa ... supplier7@demo.sa	Demo@2026

Dev mode conveniences: mock payment gateway page at `/api/v1/payments/mock/checkout/{id}`; OTP codes returned in the API response (`debug_code`) while providers are "console". Both are disabled in production by env variables.

6. Going live — the developer's first week

Follow **DEPLOYMENT.md** in the repository; it has the exact commands and a checklist. Summary:

1. **Server + domain:** Ubuntu server with Docker; DNS A record for the domain and `www . git clone , cp .env.production.example .env , fill it, docker compose -f docker-compose.prod.yml up -d --build . HTTPS is automatic (Caddy).`
2. **Secrets:** `JWT_SECRET` (openssl rand -hex 32), `POSTGRES_PASSWORD` , `ADMIN_PASSWORD` (change after first login), `SEED_DEMO_DATA=0` , `OTP_REQUIRED=1` , `OTP_DEBUG_RETURN_CODE=0` , `CORS_ORIGINS` = your domain.
3. **Moyasar:** live keys; webhook to `/api/v1/payments/webhook/moyasar` with a secret token; enable mada/cards/Apple Pay/STC Pay; do one real small payment, then a refund, and check Admin → Finance.
4. **Unifonic** (SMS): app SID + sender-ID registration through CST; **SMTP:** credentials + SPF/DKIM; verify OTP arrives on a real phone and email.
5. **Storage:** `STORAGE_BACKEND=s3` with a bucket (AWS me-south-1, OCI Jeddah, or Wasabi) for logos, product images, documents.
6. **Mobile:** `eas init` , set `apiBase` to the production URL, `eas build` for Android and iOS, test push notifications on real devices, submit to stores with the assets in `mobile/assets/` .

7. **Operations:** nightly backup cron (`deploy/backup.sh`), uptime check on `/api/v1/health` , `SENTRY_DSN` for error monitoring.
8. **Content:** review categories/products in Admin, remove demo data, invite the first real suppliers, add external price sources.

Accounts only the owner can open: Moyasar (merchant), Unifonic (sender ID needs the company's CR), SMTP/Google Workspace, Apple Developer (SAR ~370/yr), Google Play (one-time USD 25), Expo/EAS, cloud provider, Sentry (free tier). ZATCA e-invoicing onboarding and the company's VAT number are also owner tasks.

7. Remaining work — prioritised backlog

Each item: what, why, where in the code, how, and what "done" means. Estimates are for one experienced developer.

P0 Before launch (1–2 weeks)

Item	Details	Est.
Live-provider verification	The Moyasar, Unifonic/Twilio, SMTP, Meta WhatsApp and Expo push clients follow the public APIs but were only exercised with mock/console providers. Run each once with real credentials: <code>services/payments.py</code> (create_checkout, fetch_status, refund, webhook payload shape), <code>services/channels.py</code> (each provider's send), <code>mobile/src/push.ts</code> . Done when: one paid+refunded order, one OTP SMS, one email, one push received on a device.	2 d
Mobile on real devices	Run on iOS and Android via Expo Go and an EAS development build; fix layout/RTL issues (React Native RTL requires <code>I18nManager.forceRTL</code> + reload — currently only <code>allowRTL</code>); test payment return flow (app should refresh orders after returning from the browser: call <code>/payments/{id}/sync</code> on focus). Add mobile parity for v1.2 features (BOQ import, documents upload, disputes) if wanted. Done when: TestFlight + internal testing track builds pass a full buyer and supplier flow.	3–5 d
Private document storage	Supplier documents are stored with random names but publicly readable URLs. Move documents to a private bucket and serve them through an authenticated endpoint that returns a signed URL (boto3 <code>generate_presigned_url</code>). Files: <code>services/storage.py</code> (add <code>save_private/signed_url</code>), <code>routers/suppliers.py</code> and <code>routers/admin.py</code> document endpoints; web links use the new endpoint.	1 d
Multi-replica safety	Rate limiting is in-memory and the job worker runs in-process. Fine for one API container. For >1 replica: move counters to Redis (<code>main.py rate_limit</code>) and run jobs from a single cron host (<code>python -m app.jobs</code> , <code>JOBS_ENABLED=0</code> on replicas) or add a Redis lock in <code>services/jobs.py tick()</code> .	1 d
Security pass	Set strict CORS, review token lifetime (7 days; consider refresh tokens), add <code>Content-Security-Policy</code> in <code>deploy/Caddyfile</code> , enable PostgreSQL SSL if managed, run an automated scan (OWASP ZAP) against staging, confirm no demo accounts in production (<code>SEED_DEMO_DATA=0</code>), rotate any keys used during testing.	1–2 d
Legal & content	Terms of service, privacy policy (PDPL), supplier agreement (fee %, payout terms, dispute policy) as static pages (add routes in <code>web/src/App.tsx</code>); ZATCA registration; final mission/slogan copy in <code>web/src/i18n.tsx</code> (keys <code>mission</code> , <code>tagline</code>).	owner + 0.5 d
Monitoring & backups tested	Sentry DSN set; uptime monitor; run <code>deploy/backup.sh</code> and a restore on a scratch database once.	0.5 d

P1 First 3 months after launch

Item	Details	Est.
ZATCA phase-2 e-invoicing	Phase-1 QR is implemented (<code>services/payments.py</code> <code>zatca_tlv/issue_invoices</code>). Phase 2 needs UBL XML, signing with the company's CSID, and clearance/reporting via the ZATCA API. Generate XML in <code>issue_invoices</code> , store the cleared XML/hash on <code>Invoice</code> , and add a job for reporting. Consider a certified provider SDK to shorten this.	2–3 w
Buy-now-pay-later & second gateway	Tabby/Tamara for buyers and HyperPay as a fallback gateway: each is one provider class in <code>services/payments.py</code> implementing <code>create_checkout</code> / <code>fetch_status</code> / <code>refund</code> , plus a webhook route in <code>routers/payments.py</code> and a method option in <code>schemas.CheckoutIn</code> .	1 w each
Automated supplier payouts	Replace manual "mark paid" with bank API transfers (e.g. Al Rajhi/ANB corporate APIs or a payout provider). Hook in <code>routers/admin.py</code> <code>mark_payout_paid</code> and a job that batches pending payouts.	2 w
Search & catalog quality	PostgreSQL full-text search (tsvector on <code>name_ar/name_en/brand</code> ; <code>services/pricing.search_products</code> currently uses ILIKE), product de-duplication tools in Admin (merge products, remap offers), unit normalisation (ton ↔ bag ↔ m³ conversion table on <code>Product.spec</code>), synonyms for Arabic spellings.	2 w
Supplier connectors	Google Sheets: a published-CSV link already works as a price source; add a guided UI. ERP adapters (Odoo, SAP B1) as new <code>PriceSource.kind</code> values in <code>services/ingestion.py</code> ; per-supplier CSV templates; scheduled fetch is already in place.	1–2 w
Buyer teams & projects	Multiple users per buyer company (a <code>Company</code> model and membership like the supplier/owner link), project folders for RFQs and orders, PDF export of bid comparisons (render the Excel data to HTML → PDF like the invoice page).	2 w
Trust & quality	Document expiry reminders (job over <code>SupplierDocument.expires_at</code>), "verified documents" badge on supplier cards, review moderation, supplier response-time metrics on RFQs.	1 w
WhatsApp templates & bot	Meta requires approved templates for messages outside 24 h; add template names per notification kind in <code>services/channels.MetaWhatsApp</code> . Optional WhatsApp price-lookup bot (webhook → <code>search</code> → <code>reply</code>).	1 w
Growth & analytics	Product/city SEO landing pages (server-rendered or pre-rendered from the API), sitemap, GA4/Mixpanel events, referral credits, supplier featured placement tied to <code>Supplier.plan</code> .	1–2 w

P2 Later

- Price forecasts and "buy now / wait" signals per category; city arbitrage view; public weekly price index.
- Logistics marketplace (delivery partners, tracking), financing partner integration, GCC expansion (multi-currency, country field on suppliers/offers).
- Native modules for the app (camera BOQ scanning, offline price cache), tablet layouts.

8. Known limitations and technical notes

- **SQLite is for development only**; production uses PostgreSQL through `DATABASE_URL`. Schema is managed with Alembic; after any model change: `alembic revision --autogenerate`.
- **Money is stored as floats** rounded to 2 decimals, adequate for SAR amounts at this scale; switch to `Numeric(14,2)` when adding ledgers/accounting exports.
- **Escrow is logical**: funds sit in your Moyasar/bank account and payouts are tracked in the platform; use a separate settlement bank account for client money and reconcile with Admin → Finance.
- **Invoices** are ZATCA phase-1 (QR). Buyer VAT number is not captured yet (add to `User` /company when implementing teams).
- **External price feeds** expect CSV/JSON with the documented columns; scraping other websites is intentionally not included (legal/ToS); ask suppliers for feeds or use published sheets.
- **Notifications** are sent by a thread in the API process; on restart, queued rows remain in the database and are picked up again.
- **Tests** cover the backend end to end; the web app was verified by rendering real flows in Chromium during development but has no automated UI tests yet (Playwright is a natural fit — see the flows in `DEPLOYMENT.md`). Mobile has type checks only.
- **Repository**: the branch `claude/building-materials-ecommerce-ktfxj3` contains a separate, unrelated implementation from another session with failing CI. Deploy from `main`; that branch can be deleted.
- **Demo data**: prices and history are synthetic; real prices come from suppliers and feeds.

9. Where to read more

File	Content
README.md	Overview, features, run instructions, demo accounts
DEPLOYMENT.md	Production deploy step by step, provider setup, app stores, checklist
docs/ARCHITECTURE.md	Domain model, pricing logic, security, scaling path
docs/API.md	Endpoint reference (interactive docs at /docs)
docs/INTEGRATIONS.md	Payments, OTP, channels, jobs, storage, monitoring — every env variable
docs/BUSINESS_MODEL.md · docs/ROADMAP.md	Revenue lines, KPIs, fundraising ladder; shipped vs next
brand/README.md	Logo, slogan usage, colors, fonts

Support: the code is fully commented; every service has a module docstring explaining its flow. The demo dataset and tests double as executable documentation of every business rule.

Build for Less.